

28.11.2024г.

Итератори и генератори

Лекция 15

Итерация

- когато имаме някакви данни, обикновено искаме да ги обходим (с for примерно)
- итерируеми:
 - list, • set, • dict, • tuple,
 - map и filter създават итерируеми обекти
 - всичко в collections
 - всеки обект от клас имплементиращ `__getitem__`

Итерируемост (iterable) притежава `__iter__` метода

Итератор (iterator) притежава `__next__` метода

// Разликата е в това колко пъти можем да итерируем през обекта

`__iter__`

- грижи се да конструира обект, който може да бъде итерируен с for
- дефинира свойството "итерируемост" на даден обект

`iter(__iter__)` - създава итератора

`next(__next__)` - връща следващата стойност

`__next__`

- грижи се за връщане на следващата стойност в стъпката на итерация
- дефинира поведението на итератора

StopIteration

- изключение, което индикира "край" на итерираният обект

map и filter можем да итерируем само по веднъж

iter и next - Обобщено

- iter извиква __iter__ метода на аргумента си
- Ако няма, конструира итератор като извиква __getitem__ с последователни естествени числа, започвайки от нула
- Ако има — изпълнява __next__ на върнатия обект (итератор)
- докато не се хвърли грешка

— iter метода има и втора версия, в която имаме втори аргумент — sentinel. Итерацията спира, когато достигнем точно тази стойност

Генератори

- те също са итератори
- не пазят стойностите в паметта, а ги генерират когато е необходимо
- вместо return — yield

yield

- връща обект
- __iter__ и __next__ се имплементират скрито
- когато функцията yield-не, не е терминирана, а паузирана
- когато функцията приключи, StopIteration автоматично

yield from — извикване рекурсивно

map и filter

- приемат итерувими и връщат итератори

enumerate

ertools

- има методи подходящи за комбинаторика
- след като са итерирани веднъж, те се изчерпват

`itertools.repeat(objects[, times])`

шрифтът в `[]` означава, че този аргумент, може да не се подава на метода